Project Documentation
Sample Project Documentation

Date	11 July 2026
Team ID	SWTID-2026-9508
Project Name	Credit Card Approval Prediction System
Maximum Marks	4 Marks

1. Project Overview

The **Credit Card Approval Prediction System** is an end-to-end Machine Learning web application that predicts whether a credit card application should be Approved or Rejected, based on applicant demographic and financial information. It was built as part of the SmartBridge AI/ML Internship by Team SWTID-2026-9508 (K Akshaya - Team Lead, Naveen Kumar Kayala, Manogna Reddy).

2. Problem Statement

Manual credit card approval processes in banks are slow, inconsistent across officers, and prone to human bias - often taking days to evaluate an applicant's creditworthiness. This project replaces that manual process with a fast, consistent, data-driven prediction system.

3. Objectives

- Build and compare multiple ML classification models on real credit approval data.
- Select the best-performing model based on rigorous evaluation metrics.
- Deploy the model through a usable Flask web application.
- Reduce manual review time and inconsistency in the approval process.

4. Technology Stack

Layer	Technology
Language	Python 3
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Machine Learning	Scikit-learn, XGBoost
Web Framework	Flask
Frontend	HTML, CSS, Jinja2 templates
Model Persistence	Joblib
Version Control	Git, GitHub

5. Dataset

The **UCI Credit Approval dataset** was used - 690 real, anonymized credit card application records with 15 demographic and financial features and a binary target (Approved/Rejected). This is a widely-used, real-world benchmark dataset for credit scoring research.

6. Methodology

6.1 Data Preprocessing

- Missing values imputed (mean for numeric, mode for categorical fields).
- Duplicate records removed.
- Categorical features label-encoded; low-value features (DriversLicense, ZipCode) dropped.
- Numeric features scaled using StandardScaler.
- 80/20 stratified train-test split.

6.2 Model Training & Comparison

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.891	0.859	0.902	0.880
Decision Tree	0.877	0.833	0.902	0.866
Random Forest	0.906	0.929	0.852	0.889
XGBoost (Selected)	0.906	0.900	0.885	0.893

XGBoost was selected as the deployment model based on the highest F1-Score, which balances Precision and Recall - both false approvals and false rejections carry real business cost in a credit decision context.

7. System Architecture

The system follows a layered architecture: the applicant interacts with an HTML/Bootstrap form (Presentation Layer), submitted data is handled by the Flask server (Application Layer), which passes it through the saved preprocessing pipeline and trained model (Data/Model Layer) to generate a prediction that is returned to the user. Full architecture diagrams are provided in the Project Design Phase documentation (Solution Architecture.pdf and Technology Stack.pdf).

8. Testing Summary

The application was tested end-to-end using Flask's test client, simulating real HTTP requests. All 7 functional test cases (form loading, input validation, error handling, successful predictions, 404 handling) and all 3 performance test cases (response time, repeated request stability, model load time) passed. Average prediction response time was 5.4 milliseconds - well within the 3-second target. Full results are documented in Performance Testing.pdf.

9. Results

- Best model (XGBoost) achieved 90.6% test accuracy and 0.893 F1-Score.
- Flask application successfully serves live predictions with confidence scores.
- All functional and performance tests passed.
- Fully documented, professional GitHub repository covering Ideation through Testing.

10. Conclusion

The Credit Card Approval Prediction System successfully demonstrates a complete, working replacement for slow, inconsistent manual credit review - combining a rigorously compared ML model with a functional, professionally designed web application. The project meets its original objectives: faster decisions, consistent evaluation criteria, and a resume-ready, evaluation-ready deliverable.

11. Future Enhancements

- Add SHAP/LIME-based explainability so applicants and officers can see which factors drove a decision.
- Integrate with a live database to store application history and support an officer dashboard.
- Deploy to IBM Cloud or another cloud platform for public accessibility beyond local testing.
- Add authentication so only authorized bank staff can access the admin-facing parts of the system.

12. References

UCI Machine Learning Repository - Credit Approval Dataset; Scikit-learn documentation; XGBoost documentation; Flask documentation.